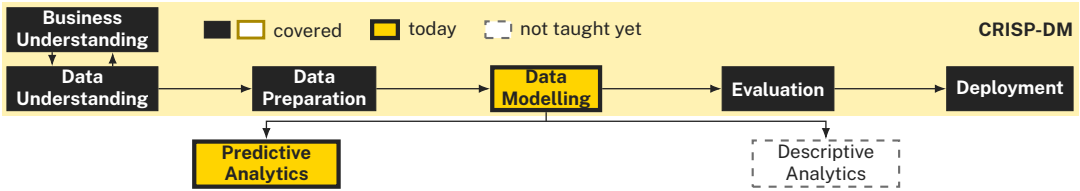
DALHOUSIE UNIVERSITY

Predictive Analytics

Targets, predictors, and classification metrics

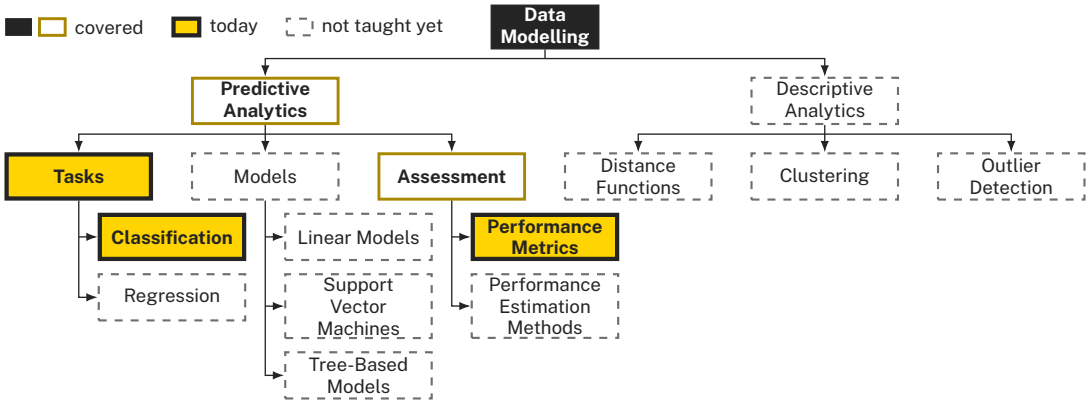
Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

Where Predictive Analytics Sits



Today we enter *Data Modelling*, where predictors are used to estimate a defined target.

The Predictive Analytics Branch



Tasks and classification metrics.

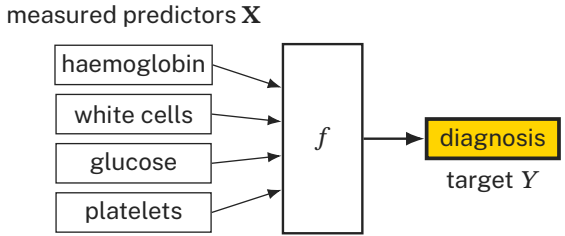
Predictive Analytics

DALHOUSIE UNIVERSITY

Use the variables you can measure to estimate the one you cannot.

Predicting One Variable from Others

In many applications it is useful to predict one variable using another, or a set of other variables.



predictors: cheap, plentiful, already recorded
target: expensive, delayed, or unknown

Everyday Prediction Examples

DALHOUSIE UNIVERSITY

EXAMPLES

- Diagnose a patient based on their blood sample results.
- Predict the ranking of a video game player based on their location and the hours they played.
- Detect a fraudulent transaction based on the time, amount transferred and location.

Target and Predictors

DALHOUSIE UNIVERSITY

THE TARGET Y

- Called the **target** variable, **response** variable, the **outcome**, or sometimes the **dependent** variable, in classification, the **label** variable.
- May be more important, more difficult to obtain, and/or influenced directly or indirectly by the other variables we measured.

THE PREDICTORS $\mathbf{X} = X_1, \ldots, X_p$

- Called the **predictors**, descriptors, explanatory variables, features, attributes, covariates.
- May be easier to obtain, and we suspect they describe the observations in a way that somehow "explains" the target variable.

The names differ by discipline, not by meaning. Statistics says response and covariates; machine learning says label and features; the course uses target and predictors.

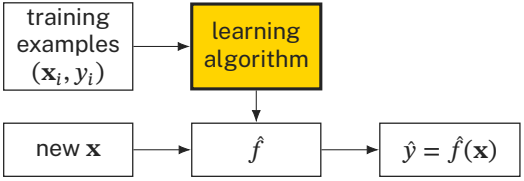
Predictive Modelling

We assume there is a relationship

$$Y = f(X_1, X_2, \ldots, X_p)$$

- **Given:** examples of observations where we measured Y and $\mathbf{X}$.
- **Goal:** find a model that approximates the unknown function f.
  - A prediction model is any algorithm that estimates f.

Since we use examples of the mapping of $\mathbf{X}$ to Y to "learn" the function f, training a model to accomplish this task is also called **supervised learning**.



the supervision is the y_i in the training examples

$\hat{f}$ is the estimate of the unknown f.

Why Learn a Model?

Predictive models have two main uses.

1. Prediction

- Make predictions of outcome given the descriptors for new observations.
- It may not be as important that the functional form of the model is easily understandable by humans.

2. Inference

- Use the models to better understand the factors that influence the outcome.
- Here obtaining the approximation of f is the most important outcome.

The two uses pull in opposite directions, and they pull on the metric too. A prediction goal is scored on its error on cases not used to build the model. An inference goal is scored on whether the fitted form is stable and readable.

Types of Modelling Tasks

The type of modelling task depends on the type of target variable Y.

Classification

Task if Y is **categorical** (nominal or ordinal).

Example: predict the species of a flower given measurements of its parts.

Regression

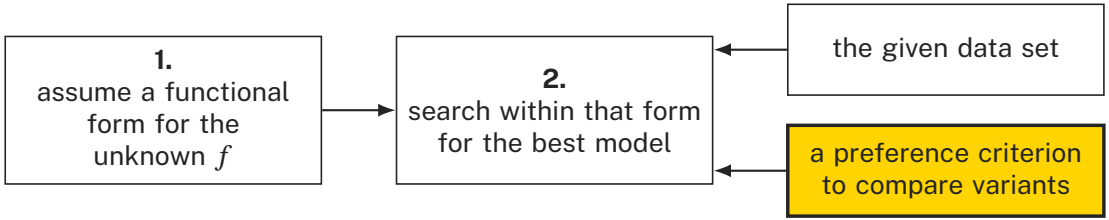
Task if Y is **numeric**.

Example: predict the market value of a house given its characteristics.

The gold box is the one this class evaluates. Everything after the section break assumes a categorical target.

Model Assumptions

There are many algorithms that can be used to approximate f based on a data set. All have some key characteristics.



The preference criterion is where the evaluation metric enters, before fitting any model.

Different Types of Models

Examples of model families.

Mathematical formulae	e.g. linear discriminants
Logical approaches	e.g. classification or regression trees, rules
Distance-based approaches	e.g. k-nearest neighbours
Other approaches	e.g. support vector machines
Sets of models (ensembles)	e.g. random forests, adaBoost

For classification evaluation, each classifier produces a predicted class for each test case. That is why one metric section serves all of them.

Pros and Cons

Different models differ in four respects.

- Assumptions on the unknown form of function f
  - which leads to different levels of flexibility in how they can approximate (or not) different types of functions;
- Computational complexity of the search problem;
- Amounts of data needed to achieve reasonable performance;
- Interpretability of the resulting model.

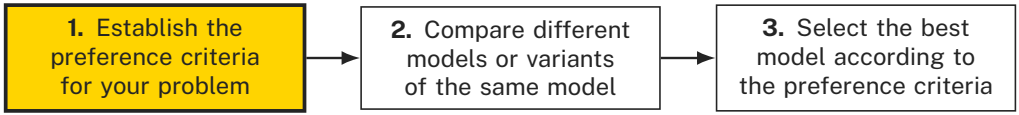
Qualitative tendencies

	Linear model	Flexible ensemble
flexibility	lower	higher
data needed	lower	higher
compute	lower	higher
interpretability	higher	lower

Schematic comparison, not measured scores. The task and implementation matter.

Which Model Should I Use?

This question is often known as the **Model Selection** problem. The answer depends on the goals of the final user.



Step 1 is the rest of this class. A metric chosen after the results are in can make any model the best.

Evaluation Metrics

DALHOUSIE UNIVERSITY

For classification problems

Every metric up to the ROC curve reads one table of counts.

The Running Ten-Case Test Set

A three-class model is evaluated on ten test cases. These ten rows anchor the metrics; a separate six-case comparison is labelled later.

case	1	2	3	4	5	6	7	8	9	10
true y_i	c1	c1	c2	c1	c3	c1	c2	c3	c2	c3
predicted $\hat{y}_i$	c1	c2	c1	c3	c3	c1	c1	c3	c1	c2
correct?	yes	no	no	no	yes	yes	no	yes	no	no

Four of the ten predictions are right. The error rate needs only that count.

CLASS BALANCE

class	Observed	Predicted
c1	4	5
c2	3	2
c3	3	3
total	10	10

The true counts are fixed by the data, and the predicted counts are chosen by the model. Almost every metric below is a ratio involving one or the other.

Classification Error, the Error Rate

Given a test set for which we know the true class, the error rate is

$$L_{0/1} = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} I(\hat{y}_i \neq y_i)$$

where $\hat{y}_i$ is the model's prediction for test case i, y_i is its true class, and I is 1 when the prediction is wrong and 0 otherwise.

$L_{0/1}$ is called *zero-one loss* because each case contributes 0 or 1 and nothing in between, a near miss and a wild miss cost exactly the same. The cost/benefit matrix later in this class drops that assumption.

The Error Rate on the Running Example

DALHOUSIE UNIVERSITY

WORKED ON THE RUNNING EXAMPLE

The indicator returns 1 on cases 2, 3, 4, 7, 9 and 10, and 0 on the other four.

$$L_{0/1} = \frac{1}{10}(6) = \frac{6}{10} = 0.6$$

Accuracy is the complement of the same count.

$$\text{Accuracy} = 1 - L_{0/1} = \frac{4}{10} = 0.4$$

Confusion Matrices

DALHOUSIE UNIVERSITY

- A square $n_c \times n_c$ matrix, where n_c is the number of class values of the problem.
- The matrix contains the number of times each pair (*ObservedClass*, *PredictedClass*) occurred when testing a classification model on a set of cases.
- The error rate follows from the counts in this table.

$$L_{0/1} = 1 - \frac{\sum_j n_{j,j}}{\sum_j \sum_k n_{j,k}}$$

		Predicted		
		c_1	c_2	c_3
Observed	c_1	$n_{1,1}$	$n_{1,2}$	$n_{1,3}$
	c_2	$n_{2,1}$	$n_{2,2}$	$n_{2,3}$
	c_3	$n_{3,1}$	$n_{3,2}$	$n_{3,3}$

Rows are the truth. Columns are the model. The diagonal $n_{j,j}$ holds the correct predictions; everything off it is an error, and the row it sits in tells you what was really there.

Which Axis Is the Truth?

Reading a confusion matrix the wrong way round swaps precision and recall. Two of the three tools follow one convention; one uses the other.

ROWS = TRUTH

```
table(trueVals, preds)
confusion_matrix(y_true, y_pred)
```

		Predicted		
		c1	c2	c3
Observed	c1	**2**	1	1
	c2	3	**0**	0
	c3	0	1	**2**

ROWS = PREDICTION

```
caret::confusionMatrix(data, reference)
```

		Observed		
		c1	c2	c3
Predicted	c1	**2**	3	0
	c2	1	**0**	1
	c3	1	0	**2**

Both tables hold the same ten cases; one is the transpose of the other. The diagonal is identical, so accuracy never notices, but precision and recall trade places. **Read the header before the cells.**

The Confusion Matrix and the Error Rate

R

```r
trueVals <- c("c1","c1","c2","c1","c3",
              "c1","c2","c3","c2","c3")
preds    <- c("c1","c2","c1","c3","c3",
              "c1","c1","c3","c1","c2")
(confMatrix <- table(trueVals, preds))
##         preds
## trueVals c1 c2 c3
##       c1  2  1  1
##       c2  3  0  0
##       c3  0  1  2
(errorRate <- 1 - sum(diag(confMatrix)) /
                  sum(confMatrix))
## [1] 0.6
```

Python

```python
from sklearn.metrics import (confusion_matrix,
                             accuracy_score)
y_true = ["c1","c1","c2","c1","c3",
          "c1","c2","c3","c2","c3"]
y_pred = ["c1","c2","c1","c3","c3",
          "c1","c1","c3","c1","c2"]
labels = ["c1","c2","c3"]
confusion_matrix(y_true, y_pred, labels=labels)
# array([[2, 1, 1],
#        [3, 0, 0],
#        [0, 1, 2]])
1 - accuracy_score(y_true, y_pred)   # 0.6
```

Both put the truth down the rows, since each takes it as the *first* argument. The diagonal holds $2, 0, 2$, four correct of ten; `c2` is never predicted correctly, which the error rate hides.

Label Order and a Transposed Table

DALHOUSIE UNIVERSITY

R | caret, for comparison

```r
library(caret)   # needs e1071 installed
lv <- c("c1", "c2", "c3")
cm <- confusionMatrix(
        factor(preds,    levels = lv),
        factor(trueVals, levels = lv))
cm$table                # PREDICTION on rows
cm$overall["Accuracy"]  # 0.4
```

Two defaults are worth knowing. `confusion_matrix` sorts labels alphabetically unless `labels=` is given, so `"yes"/"no"` comes back with `no` first. `caret::confusionMatrix` takes `data` (the predictions) first and `reference` (the truth) second, and prints them in that order.

`table()` infers observed labels from character vectors. Explicit factors retain their levels, even when absent. Use `factor(x, levels = ...)` for a fixed order and shape.

Cost-Sensitive Applications

- Error rate assigns unit cost to every mistake and zero cost to every correct prediction.
- This may not be adequate for some applications.
- Models are then evaluated by the total balance of their predictions, i.e. the sum of the benefits minus the costs.

Cost/benefit matrices

Table where each entry specifies the cost (negative benefit) or benefit of each type of prediction.

The Cost/Benefit Matrix

		Predicted		
		c_1	c_2	c_3
Observed	c_1	$B_{1,1}$	$C_{1,2}$	$C_{1,3}$
	c_2	$C_{2,1}$	$B_{2,2}$	$C_{2,3}$
	c_3	$C_{3,1}$	$C_{3,2}$	$B_{3,3}$

Benefits B sit on the diagonal, costs C off it. The matrix has the same shape and the same orientation as the confusion matrix, which is exactly why the two multiply cell by cell.

Cost and Benefit

R

```r
costMatrix <- matrix(c(10,-2,-4,-2,30,-3,-5,-6,12),
                     ncol = 3)
colnames(costMatrix) <- c("predC1","predC2",
                          "predC3")
rownames(costMatrix) <- c("obsC1","obsC2","obsC3")
costMatrix
##       predC1 predC2 predC3
## obsC1     10     -2     -5
## obsC2     -2     30     -6
## obsC3     -4     -3     12
(utilityPreds <- sum(confMatrix * costMatrix))
## [1] 28
```

Python

```python
import numpy as np
C = np.array([[10, -2, -5],
              [-2, 30, -6],
              [-4, -3, 12]])
M = confusion_matrix(y_true, y_pred,
                     labels=labels)
(M * C).sum()   # 28
```

`matrix()` fills **by column** (`byrow = TRUE` is what students usually mean). Both products are elementwise, not `%*%` or `@`, and need both matrices in the *same* orientation; `caret`'s transposed table gives 26 instead of 28.

A Better Error Rate, a Worse Decision

THE ARITHMETIC, CELL BY CELL

$$
\begin{aligned}
U &= 2(10) + 1(-2) + 1(-5) \\
&\quad + 3(-2) + 0(30) + 0(-6) \\
&\quad + 0(-4) + 1(-3) + 2(12) \\
&= 20 - 2 - 5 - 6 - 3 + 24 = 28
\end{aligned}
$$

THREE MODELS, ONE TEST SET

model	error rate	utility
Model A	**0.6**	+28
always predict c1	**0.6**	+22
always predict c2	0.7	**+73**

Two models with an *identical* error rate are worth 28 and 22. A third model with a *worse* error rate is worth 73, because $B_{2,2} = 30$ makes a correct c2 worth three correct c1s.

Always-c2: $4(-2) + 3(30) + 3(-3) = 73$.

Predicting a Rare Class

- Problems with two classes. *For example, predicting outliers.*
- One of the classes is much less frequent and it is also the most relevant.

		Predicted	
		P	N
Observed	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

To read a cell name, the *second* word is what the model said and the *first* says whether it was right. A **false negative** is a case the model called negative and was wrong about, a missed event. Rows are still the truth, columns are still the model.

What Falls Out of TP, FP, FN, TN

The running example is three-class. Collapse it to **c1 versus rest** ("is this case a c1?") and the 3×3 table becomes a 2×2 one.

		Predicted	
		c1	not c1
Observed	c1	TP = 2	FN = 2
	not c1	FP = 3	TN = 3

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{2 + 3}{10} = 0.5$$

$$\text{Error rate} = \frac{FP + FN}{TP + FP + FN + TN} = \frac{3 + 2}{10} = 0.5$$

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{3}{3 + 3} = 0.5$$

$$\text{Prevalence} = \frac{TP + FN}{TP + FP + FN + TN} = \frac{4}{10} = 0.4$$

$FP = 3 + 0$, the c2 and c3 rows of the c1 column.
$TN = 0 + 0 + 1 + 2 = 3$.
The error rate fell from 0.6 on the three-class table to 0.5. Case 10 (true c3, predicted c2) is an error between two classes the binary view merges into "not c1". Binarizing hides confusions inside the negative class.

Precision and Recall

		Predicted	
		P	N
Observed	P	TP	FN
	N	FP	TN

precision reads the **P column**
recall reads the **P row**

Precision, proportion of the signals (events) of the model that are correct.

$$Prec = \frac{TP}{TP + FP}$$

Recall, proportion of the real events that are captured by the model.

$$Rec = \frac{TP}{TP + FN}$$

Example A, a Second Data Set

		Predicted	
		P	N
Observed	P	2	2
	N	1	1

TP = 2, FN = 2, FP = 1, TN = 1, on six test cases.

$$Precision = \frac{TP}{TP + FP} = \frac{2}{2+1} = 0.667$$

$$Recall = \frac{TP}{TP + FN} = \frac{2}{2+2} = 0.5$$

$$ErrorRate = \frac{2+1}{2+2+1+1} = \frac{3}{6} = 0.5$$

The error-rate numerator is FN + FP, the two cells off the diagonal. The denominator is every case.

Call this table **Example A**. It is a *different, smaller* table from the running example, six cases rather than ten. The same recipe is then run on the running example, so the two sets of numbers can be told apart.

The Same Recipe, Running Example

c1 VERSUS REST, TEN CASES

$$Prec = \frac{TP}{TP + FP} = \frac{2}{2 + 3} = 0.4$$

$$Rec = \frac{TP}{TP + FN} = \frac{2}{2 + 2} = 0.5$$

$$Spec = \frac{TN}{TN + FP} = \frac{3}{3 + 3} = 0.5$$

Recall is 0.5 on both tables by coincidence. Precision differs because the proportion of correct c1 predictions differs.

SIDE BY SIDE

	Example A (6)	running (10)
TP / FN / FP / TN	2 / 2 / 1 / 1	2 / 2 / 3 / 3
Precision	0.667	0.400
Recall	0.500	0.500
Specificity	0.500	0.500
Accuracy	0.500	0.500
Error rate	0.500	0.500
F_1	0.571	0.444

Recall, specificity, accuracy and error rate agree by coincidence, making the switch easy to miss. From here on, the ten-case column is the one this class uses.

Precision and Recall in Code

DALHOUSIE UNIVERSITY

R | caret (needs e1071)

```r
lv <- c("c1", "rest")
tv <- factor(ifelse(trueVals == "c1", "c1", "rest"),
             levels = lv)
pv <- factor(ifelse(preds    == "c1", "c1", "rest"),
             levels = lv)
# caret prints recall as Sensitivity and
# precision as Pos Pred Value.
confusionMatrix(pv, tv, positive = "c1")
##    Sensitivity : 0.5
##    Specificity : 0.5
## Pos Pred Value : 0.4
## Neg Pred Value : 0.6
```

Python | scikit-learn

```python
from sklearn.metrics import (precision_score,
    recall_score, f1_score)
yt = ["c1" if v == "c1" else "rest"
      for v in y_true]
yp = ["c1" if v == "c1" else "rest"
      for v in y_pred]
precision_score(yt, yp, pos_label="c1")  # 0.4
recall_score(yt, yp, pos_label="c1")     # 0.5
f1_score(yt, yp, pos_label="c1")         # 0.444
```

Defaults That Differ Between R and Python

The R and Python calls return the same two numbers only because the positive class is named. Two defaults decide what you get when it is not, and the two tools name the metrics differently.

Positive class?	`caret` takes the **first factor level** unless `positive=` is given, and levels are alphabetical, so `"fraud"` loses to `"clean"`. `sklearn` defaults to `pos_label=1` and raises on string labels.
If $TP + FP = 0$	`sklearn` returns 0.0 and warns (`zero_division="warn"`); pass `zero_division=0`, `1` or `np.nan`. `caret` returns `NA`, so a never-predicted class scores 0 in one language and `NA` in the other.
Names	`caret` prints *Sensitivity* for recall, *Pos Pred Value* for precision.

F-score

DALHOUSIE UNIVERSITY

- There is a trade-off between precision and recall.
  - It is easy to improve recall if we ignore precision, and vice versa.
- Selecting a model based on two metrics is hard.
- We need a metric that captures both precision and recall.

F-score, the Formula

- F_1 is the harmonic mean of precision and recall; F_β is weighted.
- β is a user-defined value (default is 1).
- When $\beta \to 0$ the weight of recall decreases.
- When $\beta \to \infty$ the weight of precision decreases.

$$F_\beta = \frac{(\beta^2 + 1) \cdot Prec \cdot Rec}{\beta^2 \cdot Prec + Rec}$$

β^2 weights recall relative to precision inside a harmonic mean, so one number can be compared across models instead of two.

F-score, Worked on Both Tables

DALHOUSIE UNIVERSITY

WORKED AT $\beta = 1$

Running example, c1 vs rest ($P = 0.4$, $R = 0.5$)

$$F_1 = \frac{2(0.4)(0.5)}{0.4 + 0.5} = \frac{0.4}{0.9} = 0.444$$

Example A, six cases ($P = 0.667$, $R = 0.5$)

$$F_1 = \frac{2(0.667)(0.5)}{0.667 + 0.5} = 0.571$$

A harmonic mean sits near the *smaller* value. $P = 1.0$ with $R = 0.01$ scores $F_1 \approx 0.0198$, while the arithmetic mean is 0.505.

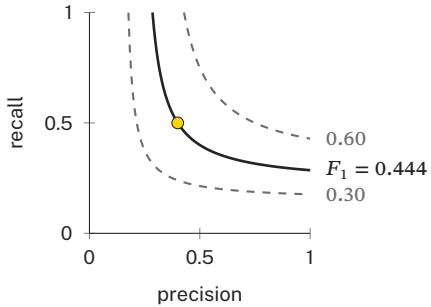
What the Weight Actually Does

Recall has relative weight β^2 in the harmonic mean. In counts,

$$F_\beta = \frac{(1+\beta^2)\,TP}{(1+\beta^2)\,TP + \beta^2 FN + FP},$$

so $\beta = 2$ weights FN by 4 and FP by 1. This is not a monetary cost ratio.

	running (10) $P = 0.400$ $R = 0.500$	Example A (6) $P = 0.667$ $R = 0.500$
$F_{0.5}$ (precision first)	0.417	0.625
F_1 (balanced)	0.444	0.571
F_2 (recall first)	0.476	0.526



Constant-F_1 curves; gold marks the running example. Raising β helps the running example, where precision is weaker, and hurts Example A, where recall is weaker.

Macro, Micro and Weighted Averaging

Precision and recall are defined one class at a time. The running example has three, so it has three of each.

class	TP	FP	FN	Prec	Rec
c1	2	3	2	0.400	0.500
c2	0	2	3	0.000	0.000
c3	2	1	1	0.667	0.667
macro				**0.356**	**0.389**
micro	4	6	6	**0.400**	**0.400**
weighted				**0.360**	**0.400**

Macro averages the three per-class values unweighted, so c2's zero counts as much as c3's 0.667. **Micro** pools the counts first, $4/(4+6) = 0.4$, exactly accuracy. **Weighted** averages by support $(4, 3, 3)$; its recall is also always accuracy.

Averaging in Code

DALHOUSIE UNIVERSITY

Python

```python
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred,
      labels=labels, digits=3, zero_division=0))
#               precision  recall  f1-score  support
#           c1      0.400   0.500     0.444        4
#           c2      0.000   0.000     0.000        3
#           c3      0.667   0.667     0.667        3
#     accuracy                         0.400       10
#    macro avg      0.356   0.389     0.370       10
# weighted avg      0.360   0.400     0.378       10
```

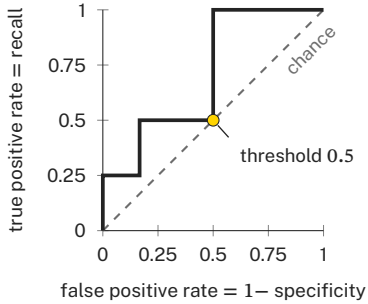
R's equivalent is `caret::confusionMatrix(...)$byClass`, which returns the per-class rows but *no* averages. You take the column means yourself and must decide macro or weighted while doing it.

One Threshold, or All of Them

Every metric so far scored one *fixed* set of predictions. Most classifiers emit a score $\hat{P}(c_1)$ and a threshold turns it into a class. Move the threshold and the whole confusion matrix moves with it.

case	1	2	3	4	5	6	7	8	9	10
true c1?	y	y	n	y	n	y	n	n	n	n
$\hat{P}(c_1)$	0.91	0.47	0.74	0.31	0.28	0.62	0.58	0.12	0.53	0.09

Cutting at 0.5 reproduces the running example's c1 column exactly, TP 2, FP 3, so recall 0.5 and $1-$specificity 0.5, the gold point, sitting on the diagonal. The *ranking*, scored across every threshold, does better, AUC $= 17/24 = 0.708$.



R's `pROC::roc` chooses direction from group medians; set `levels=` and `direction=` explicitly. Python's `roc_auc_score` keeps the supplied direction. Reversed scores give 0.292; AUC can be below 0.5.

What to Take Away

DALHOUSIE UNIVERSITY

PREDICTION MODELS

Approximate the relationship between explanatory variables and a target variable.

METRICS SELECT MODELS

Evaluation metrics are used to help select a model, and they are chosen before the comparison.

THE TARGET DECIDES THE TASK

A categorical target makes a classification problem; a numeric target makes a regression problem.

CONTEXT DECIDES THE METRIC

The most appropriate metric may depend on your application, costs, class balance, and what an error means.